

Carbon-Aware and Cost-Effective Geo-Distribution for Latency-Critical Microservices

Georgia Christofidi*
IMDEA Software Institute
Universidad Politécnica de Madrid

Francisco Álvarez-Terribas*
Telefonica Research
Universidad Pompeu Fabra

Ioannis Roumpos*
IMDEA Software Institute
Universidad Politécnica de Madrid

Nicolas Kourtellis
Keysight Technologies

Jesus Omaña Iglesias
Telefonica Research

Thaleia Dimitra Doudali
IMDEA Software Institute

Abstract

Carbon-aware workload scheduling enables significant reductions in operational carbon emissions, but existing approaches target batch, serverless, or ML workloads and assume globally distributed infrastructures. Latency-sensitive microservices deployed in regionally limited infrastructures under regulatory constraints cannot directly adopt these solutions. Our motivational analysis shows that offloading microservices to greener regions can significantly reduce carbon emissions, but only when the right microservices are selected, to meet application latency targets. These are the ones activated early in the microservice invocation call graph. Based on this insight, we design an adaptive placement framework that jointly optimizes carbon emissions, operational cost under given latency constraints. Preliminary evaluation on Kubernetes across AWS regions achieve up to 38% carbon reduction while meeting latency targets, showing that carbon-aware and cost-effective geo-distribution is possible for latency-critical microservice applications.

1 Problem Statement

Carbon-aware scheduling aims to reduce the environmental impact of cloud workloads by aligning their execution with periods or locations of lower carbon intensity. Prior work has explored two main strategies: *temporal* shifting, where workload execution is delayed to cleaner hours [5, 7, 17–19, 23, 24, 31, 33], and *spatial* shifting, where workloads are moved to greener regions [15, 23, 31, 34]. These approaches have shown substantial benefits for batch [17], HPC [18], ML training [5], and serverless workloads [15], which are naturally flexible and can tolerate delays or relocation across globally distributed infrastructures. However, these assumptions do not hold universally across cloud applications.

In particular, microservices form the backbone of many modern online services, including social networks, e-commerce platforms, financial applications, and interactive AI systems [13, 30, 37]. Unlike batch or serverless workloads, microservices are sensitive in latency, tightly coupled, and activated in structured dependency chains [11, 25, 26]. Their performance depends on fine grained RPC communication and strict Service Levels Objectives (SLOs) [29], making them far less

flexible to relocate across distant regions and impossible to delay their execution. Therefore, current carbon-aware strategies [18, 23, 33] cannot be directly applied to microservice applications.

While spatial microservice shifting (geo-distribution) is a plausible approach, there are many challenges to consider. Access to greener regions assumes global infrastructures, but that’s not the case for many organizations, especially national or regional companies and SMEs, which operate within geographically constrained infrastructures. In such cases, expanding operation to greener regions may come with additional cost and may be limited due to data sovereignty [21], privacy regulations [4, 12], and concentrated user traffic. For such organizations, carbon-aware placement must operate within a small set of nearby regions while preserving both cost efficiency and latency guarantees.

Our motivational analysis on a real world deployment shows that geo-distributing microservices can reduce both carbon emissions and operational costs. However, meeting application latency targets requires considering not only how many microservices are offloaded, but also their position in the microservice invocation graph. In particular, relocating services activated later in the execution path can incur substantial latency penalties, whereas offloading early activated services enables carbon savings with minimal impact on end-to-end latency.

2 System Overview

The space of possible service-to-region placements grows exponentially with the number of services and candidate regions. Therefore, selecting which microservices to geo-distribute, and determining their optimal placement regions, requires an informed and scalable decision-making approach.

Figure 1 shows an overview of the proposed framework, which continuously adapts microservice placement according to workload demand and regional carbon conditions. At runtime, a **Traffic Forecaster** estimates future incoming user request load. A runtime **Workload Profiler** estimates per microservice resource usage, energy consumption, latency, and the resource size that corresponds to the forecasted incoming traffic, using offline profiling information.

*PhD Students.

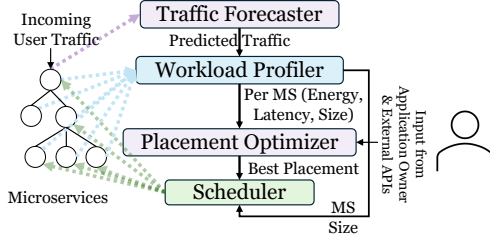


Figure 1. Proposed System Implementation.

Guided by the estimations above and pricing and carbon data from external APIs, the **Placement Optimizer**, which is the core component of our system, determines the service-to-region mapping, triggered upon workload or carbon intensity changes. It employs a genetic algorithm (GA) [14, 20] to minimize carbon emissions and monetary cost while satisfying latency constraints. Exact optimization methods are impractical due to the non linear [27, 32, 38] and combinatorial nature of the placement problem [8, 16, 28], and even GAs can struggle with the large search space. Therefore, we leverage insights from our motivational analysis to prune the search space by pinning immovable services, such as databases, and late-activated services, while excluding regions that increase carbon emissions and cost.

Finally, the **Scheduler** enacts the selected placement across cloud regions with minimal disruption. Rather than relocating running microservices, it provisions resources in the target region and instantiates the corresponding services from pre-replicated images. Once the new instances are ready to receive traffic, user requests are redirected to them and the resources allocated to the original instances are released. In microservice architectures, state is commonly externalized to databases [13, 22, 29, 37]. Our system pins heavy stateful services to the base region by design, thereby avoiding excessive data transfer costs.

3 Preliminary Results

We evaluate the framework using a Kubernetes deployment of the DeathStarBench [2, 13] social network and media streaming applications across multiple AWS regions. Across a four-day deployment with generated workload based on the Azure VM Traces dataset [36], the framework reduces operational carbon emissions by 37.4% while simultaneously lowering cloud infrastructure costs by 3.6% for a European deployment scenario, based in Frankfurt. The system chooses Paris and Stockholm to offload services, since both of them lower the carbon and cost compared to Frankfurt [1, 3]. Changes in placements are triggered by workload changes. When the load peaks, majority of the services return to Frankfurt, so as to meet the application latency targets.

Next, we use a large scale simulation to compare against baselines that represent the main classes of microservice placement strategies. Nautilus [11] serves as a single-region

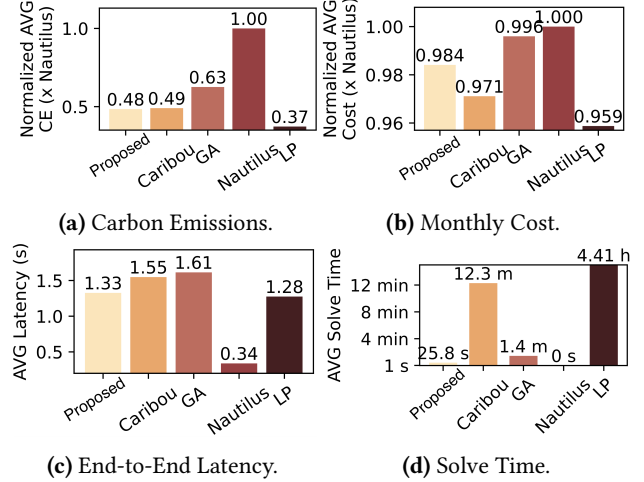


Figure 2. Comparison against baselines.

static baseline, capturing systems that optimize latency and load balancing locally but are not carbon-aware, similar to Imbres [25] and Firm [29]. Caribou [15] does carbon- and cost-aware optimization exploring spatial shifting opportunities for serverless workloads. Linear Programming (LP) provides an oracle-style exact optimization baseline, using a linear approximation of the inherently non-linear placement problem [6, 10, 35]. Finally, a vanilla genetic algorithm (GA) [9] serves as a comparison point with the algorithm used in our system. GA has no pruning enabled, exploring the full search space. Figure 2 compares the normalized carbon emissions and cost, the latency and the solve times of each approach. Our proposed system achieves carbon reductions comparable to the best performing LP optimizer, while computing placement decisions in 25.8 seconds, compared to 1.4 minutes for GA, 12.3 minutes for Caribou, and 4.4 hours for linear programming.

4 Next Steps

The preliminary results show great promise for our system. Next steps include expanding the evaluation of our system along the following directions. First, we plan to evaluate its geographic generalizability by studying carbon and cost benefits for applications deployed across diverse regions and continents (extending evaluation to US and Asia). Second, we will investigate the scalability of the framework as the number of microservices grows, reflecting larger real-world deployments. We will also do ablation and sensitivity studies. Finally, we aim to experiment with alternative resource usage and carbon intensity predictors, assessing their impact on placement quality and selecting the most accurate forecasters to anticipate incoming traffic and adapt to varying carbon conditions.

References

- [1] Accessed: 2025. AWS Pricing Calculator. <https://calculator.aws/#/createCalculator/ec2-enhancement>.

- [2] Accessed: 2025. DeathStar Benchmark. <https://github.com/delimitrou/DeathStarBench>.
- [3] Accessed: 2025. Electricity Maps. <https://app.electricitymaps.com/>.
- [4] Accessed: 2026. General Data Protection Regulation - GDPR. <https://gdpr-info.eu>.
- [5] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 118–132. <https://doi.org/10.1145/3575693.3575754>
- [6] Ivan Baev, Rajmohan Rajaraman, and Chaitanya Swamy. 2008. Approximation algorithms for data placement problems. *SIAM J. Comput.* 38, 4 (2008), 1411–1429.
- [7] Roozbeh Bostandoost, Walid A Hanafy, Adam Lechowicz, Noman Bashir, Prashant Shenoy, and Mohammad Hajiesmaili. 2024. Data-driven Algorithm Selection for Carbon-Aware Scheduling. *ACM SIGENERGY Energy Informatics Review* 4, 5 (2024), 148–153.
- [8] Claudia Canali, Caterina Gazzotti, Riccardo Lancellotti, and Felice Schena. 2023. Placement of IoT Microservices in Fog Computing Systems: A Comparison of Heuristics. *Algorithms* 16, 9 (2023). <https://doi.org/10.3390/a16090441>
- [9] Vittorio Cortellessa, Daniele Di Pompeo, and Michele Tucci. 2024. Exploring sustainable alternatives for the deployment of microservices architectures in the cloud. In *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. IEEE, 34–45.
- [10] Hao Feng, Jaime Llorca, Antonia M Tulino, Danny Raz, and Andreas F Molisch. 2017. Approximation algorithms for the NFV service distribution problem. In *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*. IEEE, 1–9.
- [11] Kaihua Fu, Wei Zhang, Quan Chen, Deze Zeng, and Minyi Guo. 2022. Adaptive Resource Efficient Microservice Deployment in Cloud-Edge Continuum. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2022), 1825–1840. <https://doi.org/10.1109/TPDS.2021.3128037>
- [12] Stanisław Galij, Grzegorz Pawlak, and Sławomir Grzyb. 2024. Modeling Data Sovereignty in Public Cloud: A Comparison of Existing Solutions. *Applied Sciences* 14, 23 (2024). <https://doi.org/10.3390/app142310803>
- [13] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 3–18. <https://doi.org/10.1145/3297858.3304013>
- [14] David E Goldberg. 1989. Optimization, and machine learning. *Genetic algorithms in Search* (1989).
- [15] Viktor Urban Gsteiger, Pin Hong Long, Yiran Sun, Parshan Javanrood, and Mohammad Shahradd. 2024. Caribou: Fine-Grained Geospatial Shifting of Serverless Applications for Sustainability. In *The 30th ACM Symposium on Operating Systems Principles (SOSP'24)*. ACM.
- [16] Lin Gu, Deze Zeng, Jie Hu, Bo Li, and Hai Jin. 2021. Layer Aware Microservice Placement and Request Scheduling at the Edge. 1–9. <https://doi.org/10.1109/INFOCOM42981.2021.9488779>
- [17] Walid A Hanafy, Qianlin Liang, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 3, Article 57 (Dec. 2023), 28 pages. <https://doi.org/10.1145/3626788>
- [18] Walid A Hanafy, Qianlin Liang, Noman Bashir, Abel Souza, David Irwin, and Prashant Shenoy. 2024. Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 479–496. <https://doi.org/10.1145/3620666.3651374>
- [19] Yankai Jiang, Rohan Basu Roy, Baolin Li, and Devesh Tiwari. 2024. EcoLife: Carbon-Aware Serverless Function Scheduling for Sustainable Computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) (SC '24). IEEE Press, Article 12, 15 pages. <https://doi.org/10.1109/SC41406.2024.00018>
- [20] Holland John. 2019. *Adaptation in natural and artificial systems*.
- [21] Vasileios Karagiannis. 2024. Data Sovereignty and Compliance in the Computing Continuum. In *2024 11th International Conference on Future Internet of Things and Cloud (FiCloud)*. 123–130. <https://doi.org/10.1109/FiCloud62933.2024.00027>
- [22] Nikita Lazarev, Shaojie Xiang, Neil Adit, Zhiru Zhang, and Christina Delimitrou. 2021. Dagger: efficient and fast RPCs in cloud microservices with near-memory reconfigurable NICs. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 36–51. <https://doi.org/10.1145/3445814.3446696>
- [23] Adam Lechowicz, Nicolas Christianson, Bo Sun, Noman Bashir, Mohammad Hajiesmaili, Adam Wierman, and Prashant Shenoy. 2025. Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints. *Proc. ACM Meas. Anal. Comput. Syst.* 9, 1, Article 8 (March 2025), 49 pages. <https://doi.org/10.1145/3711701>
- [24] Adam Lechowicz, Nicolas Christianson, Jinhua Zuo, Noman Bashir, Mohammad Hajiesmaili, Adam Wierman, and Prashant Shenoy. 2023. The Online Pause and Resume Problem: Optimal Algorithms and An Application to Carbon-Aware Load Shifting. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 3, Article 45 (Dec. 2023), 32 pages. <https://doi.org/10.1145/3626776>
- [25] Shutian Luo, Jianxiong Liao, Chenyu Lin, Huanle Xu, Zhi Zhou, and Chengzhong Xu. 2025. *Embracing Imbalance: Dynamic Load Shifting among Microservice Containers in Shared Clusters*. Association for Computing Machinery, New York, NY, USA, 309–324. <https://doi.org/10.1145/3676641.3716255>
- [26] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, Guodong Yang, and Chengzhong Xu. 2022. Erms: Efficient Resource Management for Shared Microservices with SLA Guarantees. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 62–77. <https://doi.org/10.1145/3567955.3567964>
- [27] Jinwoo Park, Byungkwon Choi, Chunghan Lee, and Dongsu Han. 2021. GRAF: a graph neural network based proactive resource allocation framework for SLO-oriented microservices. In *Proceedings of the 17th International Conference on Emerging Networking EXperiments and Technologies* (Virtual Event, Germany) (CoNEXT '21). Association for Computing Machinery, New York, NY, USA, 154–167. <https://doi.org/10.1145/3485983.3494866>
- [28] Kai Peng, Liangyuan Wang, Jintao He, Chao Cai, and Menglan Hu. 2024. Joint Optimization of Service Deployment and Request Routing for Microservices in Mobile Edge Computing. *IEEE Transactions on Services Computing* 17, 3 (2024), 1016–1028. <https://doi.org/10.1109/TSC.2024.3349408>

- [29] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 805–825. <https://www.usenix.org/conference/osdi20/presentation/qiu>
- [30] Adam Rubak and Javid Taheri. 2024. Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms. In *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing (Taormina (Messina), Italy) (UCC '23)*. Association for Computing Machinery, New York, NY, USA, Article 24, 8 pages. <https://doi.org/10.1145/3603166.3632165>
- [31] Thanathorn Sukprasert, Abel Souza, Noman Bashir, David Irwin, and Prashant Shenoy. 2024. On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud. In *Proceedings of the Nineteenth European Conference on Computer Systems (Athens, Greece) (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 924–941. <https://doi.org/10.1145/3627703.3650079>
- [32] Da Sun Handason Tam, Huanle Xu, Yang Liu, Siyue Xie, and Wing Cheong Lau. 2025. FastPERT: Towards Fast Microservice Application Latency Prediction via Structural Inductive Bias over PERT Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 19 (Apr. 2025), 20787–20795. <https://doi.org/10.1609/aaai.v39i19.34291>
- [33] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. 2021. Let's wait awhile: how temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference* (Québec city, Canada) (*Middleware '21*). Association for Computing Machinery, New York, NY, USA, 260–272. <https://doi.org/10.1145/3464298.3493399>
- [34] Li Wu, Walid A. Hanafy, Abel Souza, Khai Nguyen, Jan Harkes, David Irwin, Mahadev Satyanarayanan, and Prashant Shenoy. 2025. CarbonEdge: Leveraging Mesoscale Spatial Carbon-Intensity Variations for Low Carbon Edge Computing. *arXiv:2502.14076 [cs.DC]* <https://arxiv.org/abs/2502.14076>
- [35] Song Yang, Fan Li, Stojan Trajanovski, Xu Chen, Yu Wang, and Xiaoming Fu. 2019. Delay-aware virtual network function placement and routing in edge clouds. *IEEE Transactions on Mobile Computing* 20, 2 (2019), 445–459.
- [36] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 724–739. <https://doi.org/10.1145/3477132.3483580>
- [37] Yanqi Zhang, Zhuangzhuang Zhou, Sameh Elnikety, and Christina Delimitrou. 2024. Ursa: Lightweight Resource Management for Cloud-Native Microservices. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 954–969. <https://doi.org/10.1109/HPCA57654.2024.00077>
- [38] Zhizhou Zhang, Murali Krishna Ramanathan, Prithvi Raj, Abhishek Parwal, Timothy Sherwood, and Milind Chabbi. 2022. {CRISP}: Critical path analysis of {Large-Scale} microservice architectures. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 655–672.